

Data Encryption Standard (DES)

What is DES?

Data Encryption Standard (DES) refers to a symmetric key encryption algorithm that was developed by the National Institute of Standards and Technology (NIST) in 1977 as an official standard for encrypting data. In symmetric-key algorithms, the same keys are used for both encryption and decryption thus, secure communication heavily relies on proper key management.

DES operates on blocks of 64 bits using a key length of 56 bits. There are 16 rounds where substitution and permutation steps are applied to convert plaintext into ciphertext during each round of processing done by the algorithm. Although AES has replaced it with stronger methods such as triple DES (3DES), some old systems still use this technology because it remains significant within cryptographic history.

Implementing DES Algorithm (For Encryption) in Cryptography

To understand the DES algorithm, let's use a simple example. We'll start with a 64-bit block of plaintext and a 56-bit key.

Initial Permutation (IP)

The first step in DES is the Initial Permutation (IP). This step rearranges the bits of the plaintext based on a predefined table.

Example:

- Plaintext (in Hexadecimal): 0123456789ABCDEF
- Plaintext (in binary): 00000001 00100011 01000101 01100111 10001001 10101111 11001101 11101111

Initial Permutation Table:

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

To perform the initial permutation, we rearrange the bits of the plaintext according to the table. For instance, the bit in position 58 of the original plaintext moves to position 1 of the permuted text.

Permutation example:

- Bit at position 58 (original) → Bit at position 1 (permuted)
- Bit at position 50 (original) → Bit at position 2 (permuted)
- Continue for all 64 bits...

After Initial Permutation (example): 11001100 11110000 10101010 11001100 10101010 11001100 11110000 11001100

Splitting

Next, we split the permuted plaintext into two halves. Each half is 32 bits.

- **Left half (L0):** 11001100 11110000 10101010 11001100
- **Right half (R0):** 10101010 11001100 11110000 11001100

Key Generation

The initial 64-bit key is transformed into a 56-bit key by discarding every 8th bit. This process reduces the key to 56 bits, which is then divided into two 28-bit halves.

Example:

- Initial Key (in hex): 133457799BBCDFF1
- Convert to binary and drop every 8th bit
- Key (in binary): 00010011 00110100 01010111 01111001 10011011 10111100 11011111 11110001

Drop every 8th bit:

- 56-bit Key: 00010010 01101001 01011011 11001001 10110111 10110111 11110001

48-bit Key Generation Process for Each Round

Splitting into Halves

The 56-bit key is divided into two 28-bit halves:

- C0 (first 28 bits): 00010010 01101001 01011011 1100
- D0 (second 28 bits): 1001 10110111 10110111 11110001

Shifting

For each of the 16 rounds, the halves are circularly shifted left by one or two positions, depending on the round number.

Number of shifts per round:

Round	Shifts	Round	Shifts
1	1	9	1
2	1	10	2
3	2	11	2
4	2	12	2
5	2	13	2
6	2	14	2
7	2	15	2
8	2	16	1

Example for Rounds 1 and 2:

Round 1:

- Shift C0 and D0 left by 1 position
- C1: 00100100 11010010 10110111 1000
- D1: 00110111 01111001 10111111 1111

Round 2:

- Shift C1 and D1 left by 1 position
- C2: 01001001 10100101 01101111 0000
- D2: 01101110 11110011 01111111 1110

This shifting process is repeated for all 16 rounds.

Compression Permutation

After shifting, the 56-bit key (combined C and D halves) is compressed to 48 bits using a permutation table. This table selects 48 specific bits out of the 56-bit combined key to create the 48-bit round key.

Compression Permutation Table:

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

Example:

- Combined C1 and D1: 00100100 11010010 10110111 10000011 01111001 10111111
- Apply the compression permutation to select and rearrange the bits
- 48-bit Round Key (example): 111000110010001010110010000100110011010000101111

Rounds (Round 1 as an Example)

DES involves 16 rounds of key transformations. For each round, a 48-bit sub-key is generated from the 56-bit key.

After splitting the blocks, we use the round function. In each round, the left half remains as it is, and we need to perform the operations below on the right half:

1. Expansion Permutation (E)

2. Key mixing ($\oplus$)
3. Substitution (S1, S2,...,S8)
4. Permutation (P)

After getting the final result from the right half, we need to perform an XOR operation of the left and right half.

Expansion Permutation (E)

The 32-bit right half (R0) is expanded to 48 bits using the expansion table. The expansion table duplicates certain bits to increase the size from 32 to 48 bits.

Expansion Table:

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Apply the expansion permutation on R0:

- R0 (32 bits): 10101010 10101010 10101010 10101010
- Expanded R0 (48 bits): 01010101 01011010 10100101 01101010 01011010 10101010

Key Mixing ($\oplus$)

The expanded R0 is XORed with the round key (48-bit).

- Round Key (48 bits): 111000110010001010110010000100110011010000101111
- Perform XOR operation
- Result: 10110101 00010000 00011111 01111000 01010000 00011101

Substitution (S-Boxes)

The result is divided into eight 6-bit blocks, and each block is substituted using the S-Boxes.

Each 6-bit block is used to look up the corresponding 4-bit output from the S-Box.

Example using first 6-bit block:

- First 6-bit block: 101101
- Row: 11 (first and last bits)
- Column: 0110 (middle four bits)
- Output (S1[11][0110]): 0100

Perform this for all 8 blocks and combine the results to get a 32-bit block.

Permutation (P)

The 32-bit result of the S-Boxes is permuted using a fixed table:

Permutation Table:

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

Applying this permutation provides the final 32-bit output of the round function (F).

XOR Operation

After getting the final result from the right half, we need to perform an XOR operation of the left half (L0) and the result from the right half.

- Final 32-bit output from F: <32-bit result from P>
- L0 (32 bits): 11001100 00000000 11001100 11110000
- XOR Result: <L0 XOR 32-bit result from F>

Result after Round 1 (Swapping required)

The result after Round 1 is the combination of the original right half (R0) and the XOR result:

- L1: <Original R0>
- R1: <XOR Result>

This process is repeated for all 16 rounds. Each round uses a different 48-bit sub-key, and the final result is obtained after the 16th round, followed by the final permutation (FP).

By following these steps, the plaintext is transformed into ciphertext using DES.

Implementing DES Algorithm (For Decryption)

Just like encryption, the decryption process in DES involves 16 rounds and uses the same steps but in reverse order.

Initial Permutation (IP)

The first step in decryption is the Initial Permutation (IP). This step rearranges the bits of the ciphertext based on the same predefined table used in encryption.

Initial Permutation Table:

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

After Initial Permutation (example): 11001100 11110000 10101010 11001100 10101010 11001100 11110000 11001100

Splitting

Split the permuted ciphertext into two halves. Each half is 32 bits.

- Left half (L0): 11001100 11110000 10101010 11001100
- Right half (R0): 10101010 11001100 11110000 11001100

Round Function (Round 1)

In Round 1, we perform the following operations on the right half (R0) in reverse order of encryption:

Expansion Permutation (E)

The 32-bit right half (R0) is expanded to 48 bits using the expansion table.

Expansion Table:

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Apply the expansion permutation on R0:

- R0 (32 bits): 10101010 11001100 11110000 11001100
- Expanded R0 (48 bits): 01011010 10101100 10101100 10111100 01100110 01100101

Key Mixing ($\oplus$)

The expanded R0 is XORed with the round key (48-bit). For decryption, we use the round keys in reverse order, starting from Round 16 to Round 1.

- Example Round Key for Decryption (48 bits, Round 16):
111000110010001010110010000100110011010000101111
- Perform XOR operation
- Result of XOR operation: 10111010 01101001 00011110 10101110 10001101 01000010

Substitution (S-Boxes)

The result from the XOR operation is divided into eight 6-bit blocks, and each block is substituted using the S-Boxes.

Example:

- First 6-bit block: 101110
- Row: 10 (first and last bits)
- Column: 0111 (middle four bits)
- Output (S1[10][0111]): 1100

Perform this for all 8 blocks and combine the results to get a 32-bit block.

Permutation (P)

The 32-bit result of the S-Boxes is permuted using a fixed table.

Permutation Table:

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

Applying this permutation provides the final 32-bit output of the round function (F).

XOR Operation

After getting the final result from the right half, we need to perform an XOR operation with the left half (L0).

- Final 32-bit output from F: <32-bit result from P>
- L0 (32 bits): 11001100 11110000 10101010 11001100
- XOR Result: <L0 XOR 32-bit result from F>

Result after Round 1 (Swapping required)

The result after Round 1 is the combination of the original right half (R0) and the XOR result:

- L1: <Original R0>
- R1: <XOR Result>

Perform these operations for all 16 rounds.

Final Permutation (FP)

The final permutation rearranges the bits of the combined result from the last round using a predefined table, which is the inverse of the initial permutation table.

Final Permutation Table:

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

By applying this permutation, we obtain the final decrypted plaintext.

DES Algorithm Step-by-Step Example

Let's trace through the DES encryption with the given values:

- Plaintext: 123456ABCD132536 (hexadecimal)
- Key: AABB09182736CCDD (hexadecimal)

Step 1: Key Generation

Convert Key to Binary

Key (hex): AABB09182736CCDD
Key (bin): 101010101110110000100100011000001001110011011001100110011011101

Apply Parity Bit Drop (64 → 56 bits)

Using the keyp table, we extract 56 bits by removing parity bits:

After parity drop: 111100001100110010101010111101010101100110011110001111

Split into Left and Right Halves (28 bits each)

Left (C0): 1111000011001100101010101111
Right (D0): 0101010101100110011110001111

Generate 16 Round Keys

For each round, we shift left and compress to 48 bits:

Round 1: Shift left by 1

C1: 1110000110011001010101011111
D1: 1010101011001100111100011110
Combined: 11100001100110010101010111111010101011001100111100011110
After compression: 0001101100000010111011111111000111000001110010
Round Key 1: 0E329FFFC6FC88

Round 2: Shift left by 1

C2: 110000110011001010101010111111
D2: 0101010110011001111000111101
Round Key 2: 1A746C08CA18ADFE

This continues for all 16 rounds...

Step 2: Plaintext Processing

Convert Plaintext to Binary

Plaintext (hex): 123456ABCD132536
Plaintext (bin): 0001001000110100010101101010101111001101000100110010010100110110

Initial Permutation

Using the `initial_perm` table:

Before IP: 0001001000110100010101101010101111001101000100110010010100110110
After IP: 110011000000000011001100111111111110000101010101111000010101010

Split into Left and Right Halves

Left (L0): 11001100000000001100110011111111
Right (R0): 11110000101010101111000010101010

Step 3: 16 Rounds of Feistel Function

Round 1 Example

Step 3a: Expansion (32 → 48 bits) Expand R0 using expansion table:

R0: 11110000101010101111000010101010
Expanded: 011110100001010101010101010111101000010101010101

Step 3b: XOR with Round Key

Expanded R0: 011110100001010101010101010111101000010101010101
Round Key 1: 0001101100000010111011111111000111000001110010
XOR Result: 011000010001011110111010100001100110010100100111

Step 3c: S-Box Substitution Split into 8 groups of 6 bits and apply S-boxes:

Group 1: 011000 → S-box 1 → Row 0, Col 12 → 5 → 0101
Group 2: 010001 → S-box 2 → Row 1, Col 8 → 12 → 1100
Group 3: 011110 → S-box 3 → Row 0, Col 15 → 8 → 1000
...and so on

Combined S-box output (32 bits):

01011100100000100110010001110100

Step 3d: Permutation Apply straight permutation using `per` table:

```
Before P: 01011100100000100110010001110100
After P:  00100011010010110010100110111000
```

Step 3e: XOR with Left Half

```
L0:      11001100000000001100110011111111
P output: 00100011010010110010100110111000
New R1:   11101111010010111110010101000111
```

Step 3f: Swap

```
L1 = R0 = 11110000101010101111000010101010
R1 =      11101111010010111110010101000111
```

Continue for 15 More Rounds...

After Round 16:

```
L16: 01000011010010110010100110111000
R16: 11001100000000001100110011111111
```

Step 4: Final Processing

Combine L16 and R16

```
Combined: 0100001101001011001010011011100011001100000000011001100111111111
```

Final Permutation

Apply `final_perm` table (inverse of initial permutation):

```
Before FP: 0100001101001011001010011011100011001100000000011001100111111111
After FP:  1000101111101000000100110101010000001111000010101011010000000101
```

Convert to Hexadecimal

Final ciphertext: 85E813540F0A AB41

Decryption

For decryption, we use the same process but with round keys in reverse order:

- Use keys K16, K15, K14, ..., K2, K1 instead of K1, K2, K3, ..., K15, K16
- This reverses the encryption process and recovers the original plaintext

How DES S-Boxes Work

S-Box Structure

Each S-box is a 4×16 lookup table that converts:

- Input: 6 bits
- Output: 4 bits

DES has 8 S-boxes (S1 through S8), each handling 6 bits of the 48-bit input.

S-Box Input/Output Mapping

Input Format: 6 bits = $b_1b_2b_3b_4b_5b_6$

- Row bits: b_1b_6 (first and last bit) → determines row (0-3)
- Column bits: $b_2b_3b_4b_5$ (middle 4 bits) → determines column (0-15)

Example: S-Box 1

S1 = [[14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7],	# Row 0
[0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8],	# Row 1
[4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0],	# Row 2
[15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13]]	# Row 3

Step-by-Step S-Box Process

Scenario: First Round of DES

After expansion and XOR with round key, we have 48 bits. Let's say the result is:

```
011000010001011110111010100001100110010100100111
```

This gets split into 8 groups of 6 bits for the 8 S-boxes:

Group 1: 011000 → S-Box 1

Step 1: Extract Row and Column

```
Input: 011000
Row bits: 0__0 = 002 = 010 (Row 0)
Column bits: _1100_ = 11002 = 1210 (Column 12)
```

Step 2: Lookup in S-Box 1

```
S1[0][12] = 5
```

Step 3: Convert to 4-bit Binary

```
510 = 01012
```

Group 2: 010001 → S-Box 2

Step 1: Extract Row and Column

```
Input: 010001
Row bits: 0__1 = 012 = 110 (Row 1)
Column bits: _1000_ = 10002 = 810 (Column 8)
```

Step 2: Lookup in S-Box 2

```
S2[1][8] = 12
```

Step 3: Convert to 4-bit Binary

$$12_{10} = 1100_2$$

Continue for All 8 Groups

Group	6-bit Input	Row	Col	S-Box	Value	4-bit Output
1	011000	0	12	S1	5	0101
2	010001	1	8	S2	12	1100
3	011110	0	15	S3	8	1000
4	111010	0	13	S4	3	0011
5	100001	1	0	S5	14	1110
6	100110	0	3	S6	15	1111
7	010100	0	10	S7	6	0110
8	100111	1	3	S8	8	1000

Final S-Box Output (32 bits)

0101 1100 1000 0011 1110 1111 0110 1000
= 5C83EF68 (hexadecimal)

Why S-Boxes are Critical

1. Non-linearity

- Without S-boxes, DES would be a linear function
- Linear ciphers can be broken easily with linear algebra
- S-boxes introduce non-linear transformations

2. Confusion

- Each output bit depends on multiple input bits in complex ways
- Small input changes create large output changes

3. Size Reduction

- Compress 48 bits down to 32 bits
- Forces information loss, making reverse engineering harder

S-Box Design Criteria (Revealed in 1994)

The S-boxes were designed to:

1. Resist differential cryptanalysis
 2. Avoid linear approximations
 3. Ensure good avalanche effect
 4. Minimize correlation between input/output bits
-

DES Key Compression - Step-by-Step Process

Overview

Key compression in DES reduces a 56-bit key to 48-bit round keys through a specific permutation table. This happens **16 times** to generate 16 different round keys.

The Complete Key Generation Process

Step 1: Initial Key (64 bits → 56 bits)

Original Key: AAB09182736CCDD (64 bits)
After parity drop: 56 bits (removes 8 parity bits)

Step 2: Split into Two 28-bit Halves

C0 (Left): 11110000110011001010101111 (28 bits)
D0 (Right): 010101011100110011110001111 (28 bits)

Step 3: Generate Round Keys (16 rounds)

For each round i (1 to 16):

1. Left shift both C and D by `shift_table[i]` positions

2. **Combine** the shifted halves (56 bits total)
3. **Apply compression** using key_comp table (56 → 48 bits)

Key Compression Table Explained

The `key_comp` table specifies which 48 bits to select from the 56-bit combined key:

```
key_comp = [14, 17, 11, 24, 1, 5,      # Positions 1-6 of round key
            3, 28, 15, 6, 21, 10,      # Positions 7-12 of round key
            23, 19, 12, 4, 26, 8,      # Positions 13-18 of round key
            16, 7, 27, 20, 13, 2,      # Positions 19-24 of round key
            41, 52, 31, 37, 47, 55,    # Positions 25-30 of round key
            30, 40, 51, 45, 33, 48,    # Positions 31-36 of round key
            44, 49, 39, 56, 34, 53,    # Positions 37-42 of round key
            46, 42, 50, 36, 29, 32]    # Positions 43-48 of round key
```

Interpretation: The first round key bit comes from position 14 of the 56-bit key, the second bit comes from position 17, etc.

Detailed Example: Round 1

Input Setup

```
Key: AAB09182736CCDD
After all initial processing:
C0 = 11110000110011001010101111
D0 = 0101010101100110011110001111
```

Step 1: Left Shift

Shift both halves left by 1 position (from `shift_table[0] = 1`):

```
C1 = 111000011001100101010101111  (C0 shifted left by 1)
D1 = 101010101100110011110001110  (D0 shifted left by 1)
```

Step 2: Combine

```

Combined = C1 + D1 (56 bits total)
Combined = 1110000110011001010101011111110101011001100111100011110
              ^                               ^
            Bit 1                       Bit 29 (start of D1)

```

Step 3: Apply Key Compression

Now we select 48 specific bits using the key_comp table:

```

Position:  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 ...
key_comp: 14 17 11 24  1  5  3 28 15  6 21 10 23 19 12  4 ...

```

Bit-by-bit selection:

- Round key bit 1 = Combined[14-1] = Combined[13] = 0
- Round key bit 2 = Combined[17-1] = Combined[16] = 1
- Round key bit 3 = Combined[11-1] = Combined[10] = 1
- Round key bit 4 = Combined[24-1] = Combined[23] = 0
- Round key bit 5 = Combined[1-1] = Combined[0] = 1
- Round key bit 6 = Combined[5-1] = Combined[4] = 0
- ...and so on for all 48 bits

Step 4: Result

```

Round Key 1 (48 bits): 00011011000000101110111111111000111000001110010
Round Key 1 (hex):      0E329FFFC6FC88

```

Complete Key Schedule

Round	Left Shifts	C (after shift)	D (after shift)	Round Key (hex)
1	1	1110000110...	1010101011...	0E329FFFC6FC88
2	1	1100001100...	0101010110...	1A746C08CA18AD
3	2	0000110011...	0101011001...	FE83B4A4D1EC2D
4	2	0011001101...	0101100110...	...
...	...	...	...	...
16	1	1111000011...	0101010101...	...

Why Key Compression?

1. Security Purpose

- Reduces key material per round: Only 48 of 56 bits used in each round
- Creates different round keys: Each round gets different 48 bits
- Prevents related-key attacks: Different bit selections make keys less related

2. Algorithm Requirement

- Matches S-box input size: After expansion, we have 48 bits that need to XOR with 48-bit round key
- Fits Feistel structure: The 48-bit size works perfectly with DES's expansion function

3. Diffusion

- Spreads key influence: Each key bit influences multiple rounds
- Non-uniform selection: Some key bits used more often than others

Key Compression vs Other Operations

Operation	Input → Output	Purpose
Parity Drop	64 → 56 bits	Remove parity bits
Key Compression	56 → 48 bits	Generate round-specific keys
Expansion (data)	32 → 48 bits	Prepare data for key mixing
S-box	48 → 32 bits	Non-linear transformation